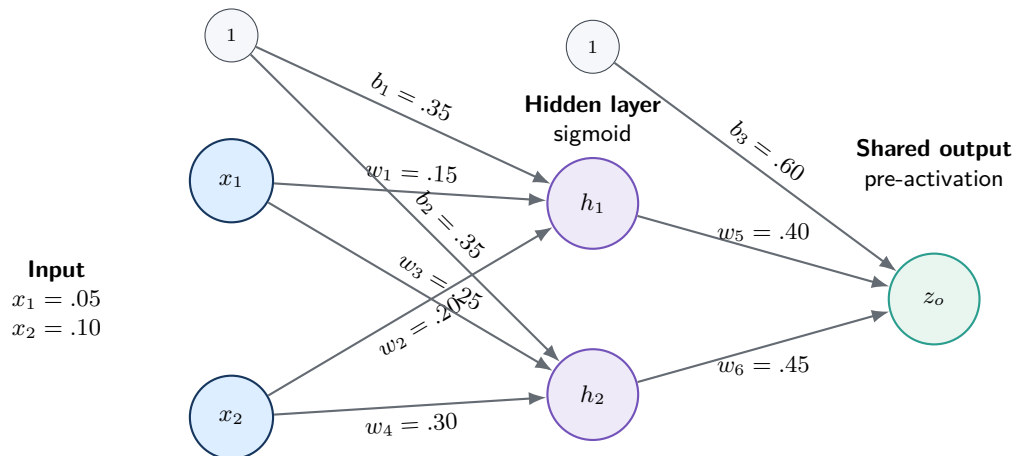


Forward and Backpropagation - One Network, Two Tasks

Regression and binary classification solved step by step with the same hidden network

What stays the same? The inputs, hidden layer, hidden activation, and chain rule.

What changes? The *output activation*, the *loss*, and therefore the *output delta*.



Regression head

Identity output:

$$\hat{y}_r = z_o$$

MSE loss:

$$E_r = \frac{1}{2}(\hat{y}_r - t_r)^2, \quad t_r = 0.50$$

Output delta:

$$\delta_o^{(r)} = \hat{y}_r - t_r$$

Binary classification head

Sigmoid probability:

$$\hat{y}_c = \sigma(z_o)$$

Binary cross-entropy:

$$E_c = -[t_c \ln \hat{y}_c + (1 - t_c) \ln(1 - \hat{y}_c)], \quad t_c = 1$$

Output delta:

$$\delta_o^{(c)} = \hat{y}_c - t_c$$

Displayed precision. Values are shown to about four decimal places so small gradients remain visible. Full precision is used in the calculations.

Step 1 - Shared Forward Pass Through the Hidden Layer

Every neuron first computes a **weighted sum** z , then applies its activation function. For sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Hidden neuron 1

$$\begin{aligned} z_{h1} &= w_1x_1 + w_2x_2 + b_1 \\ &= 0.15(0.05) + 0.20(0.10) + 0.35 \end{aligned}$$

$$z_{h1} = 0.3775$$

Then

$$h_1 = \sigma(0.3775) = 0.5933.$$

Its local sigmoid slope is

$$h_1(1 - h_1) = 0.5933(0.4067) = 0.2413.$$

Hidden neuron 2

$$\begin{aligned} z_{h2} &= w_3x_1 + w_4x_2 + b_2 \\ &= 0.25(0.05) + 0.30(0.10) + 0.35 \end{aligned}$$

$$z_{h2} = 0.3925$$

Then

$$h_2 = \sigma(0.3925) = 0.5969.$$

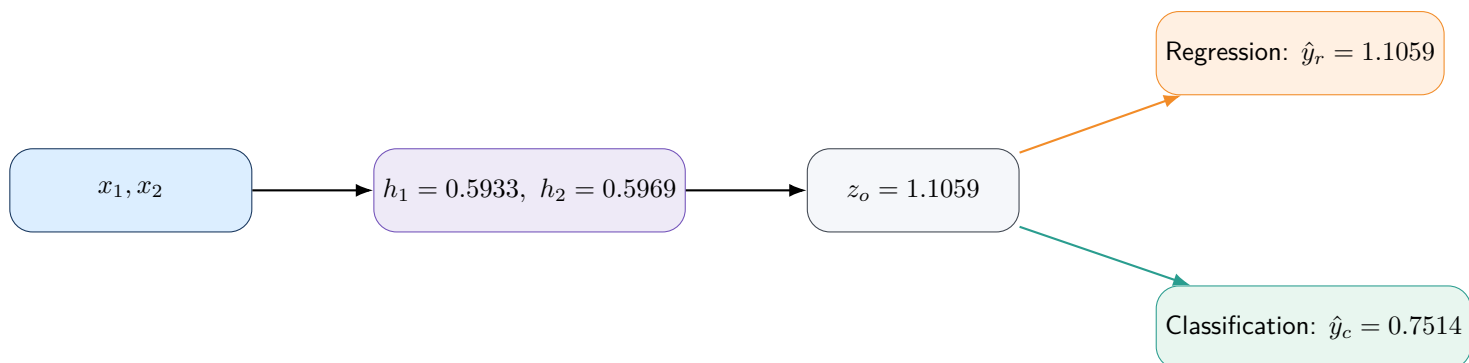
Its local sigmoid slope is

$$h_2(1 - h_2) = 0.5969(0.4031) = 0.2406.$$

Shared output pre-activation

$$\begin{aligned} z_o &= w_5h_1 + w_6h_2 + b_3 \\ &= 0.40(0.5933) + 0.45(0.5969) + 0.60 = 1.1059. \end{aligned}$$

At this point the two tasks split. Regression uses $\hat{y}_r = z_o$; binary classification uses $\hat{y}_c = \sigma(z_o)$.



Regression - Forward Pass and Output Delta

For unbounded scalar regression, use an **identity output**:

$$\hat{y}_r = z_o = 1.1059.$$

Target:

$$t_r = 0.50.$$

Therefore the prediction error is

$$\hat{y}_r - t_r = 1.1059 - 0.50 = \boxed{0.6059}.$$

Regression loss

$$\begin{aligned} E_r &= \frac{1}{2}(\hat{y}_r - t_r)^2 \\ &= \frac{1}{2}(1.1059 - 0.50)^2 \\ &= \boxed{0.1836}. \end{aligned}$$

The factor $\frac{1}{2}$ is convenient because it cancels the 2 produced by differentiating the square.

Output delta

$$\delta_o^{(r)} = \frac{\partial E_r}{\partial z_o}$$

Since $\hat{y}_r = z_o$,

$$\frac{\partial \hat{y}_r}{\partial z_o} = 1.$$

Hence

$$\delta_o^{(r)} = (\hat{y}_r - t_r)(1) = \boxed{0.6059}.$$

Interpretation of the sign. The regression prediction is too high ($1.1059 > 0.50$), so $\delta_o^{(r)} > 0$. Gradient descent will decrease positively connected output parameters such as w_5, w_6, b_3 .

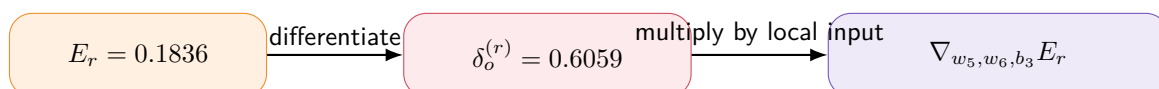
Output-layer gradients

For any output weight,

$$\frac{\partial E}{\partial w} = \underbrace{\delta_o}_{\text{output sensitivity}} \times \underbrace{\text{local input}}_{\text{activation entering the weight}}.$$

Thus

$$\begin{aligned} \frac{\partial E_r}{\partial w_5} &= 0.6059(0.5933) = \boxed{0.3595}, & \frac{\partial E_r}{\partial w_6} &= 0.6059(0.5969) = \boxed{0.3617}, \\ \frac{\partial E_r}{\partial b_3} &= \boxed{0.6059}. \end{aligned}$$



Regression - Backpropagate Through the Hidden Layer

The output delta now flows backward through w_5 and w_6 . Each hidden neuron then multiplies the incoming sensitivity by its own sigmoid slope.

1. Backpropagate to hidden activations

$$\frac{\partial E_r}{\partial h_1} = \delta_o^{(r)} w_5 = 0.6059(0.40) = \boxed{0.2424},$$

$$\frac{\partial E_r}{\partial h_2} = \delta_o^{(r)} w_6 = 0.6059(0.45) = \boxed{0.2727}.$$

2. Convert to hidden deltas

$$\delta_{h1}^{(r)} = \frac{\partial E_r}{\partial z_{h1}} = \frac{\partial E_r}{\partial h_1} h_1(1 - h_1)$$

$$= 0.2424(0.2413) = \boxed{0.0585},$$

$$\delta_{h2}^{(r)} = 0.2727(0.2406) = \boxed{0.0656}.$$

3. Input-to-hidden gradients

Neuron h_1

$$\frac{\partial E_r}{\partial w_1} = \delta_{h1}^{(r)} x_1 = 0.0585(0.05) = \boxed{0.0029}$$

$$\frac{\partial E_r}{\partial w_2} = 0.0585(0.10) = \boxed{0.0058}$$

$$\frac{\partial E_r}{\partial b_1} = \boxed{0.0585}$$

Neuron h_2

$$\frac{\partial E_r}{\partial w_3} = 0.0656(0.05) = \boxed{0.0033}$$

$$\frac{\partial E_r}{\partial w_4} = 0.0656(0.10) = \boxed{0.0066}$$

$$\frac{\partial E_r}{\partial b_2} = \boxed{0.0656}$$

One fully expanded chain for w_1 :

$$\frac{\partial E_r}{\partial w_1} = \underbrace{(\hat{y}_r - t_r)}_{\text{loss derivative}} \cdot \underbrace{1}_{\text{identity slope}} \cdot \underbrace{w_5}_{\text{output depends on } h_1} \cdot \underbrace{h_1(1 - h_1)}_{\text{hidden sigmoid slope}} \cdot \underbrace{x_1}_{\text{local input}}.$$

Numerically:

$$0.6059 \cdot 1 \cdot 0.40 \cdot 0.2413 \cdot 0.05 = \boxed{0.0029}.$$

Regression - One Gradient-Descent Update

Use learning rate

$$\eta = 0.10$$

and the standard update

$$\theta' = \theta - \eta \frac{\partial E}{\partial \theta}.$$

Parameter	Old value	Gradient	New value
w_1	0.1500	0.0029	0.1497
w_2	0.2000	0.0058	0.1994
b_1	0.3500	0.0585	0.3442
w_3	0.2500	0.0033	0.2497
w_4	0.3000	0.0066	0.2993
b_2	0.3500	0.0656	0.3434
w_5	0.4000	0.3595	0.3641
w_6	0.4500	0.3617	0.4138
b_3	0.6000	0.6059	0.5394

Recompute the forward pass

With updated parameters:

$$\begin{aligned} z'_{h1} &= 0.3716, & h'_1 &= \sigma(0.3716) = 0.5918, \\ z'_{h2} &= 0.3859, & h'_2 &= \sigma(0.3859) = 0.5953. \end{aligned}$$

Then

$$\hat{y}'_r = w'_5 h'_1 + w'_6 h'_2 + b'_3 = \boxed{1.0012}.$$

New loss:

$$E'_r = \frac{1}{2}(1.0012 - 0.50)^2 = \boxed{0.1256}.$$

Did learning move in the correct direction?

$$0.1836 \longrightarrow 0.1256$$

Yes. After one gradient step, the regression loss decreases substantially.

Binary Classification - Forward Pass and Output Delta

The hidden values and $z_o = 1.1059$ are exactly the same. Binary classification changes the output interpretation:

$$\hat{y}_c = \sigma(z_o) = \sigma(1.1059) = \boxed{0.7514}.$$

Interpretation: the model assigns about 75.14% probability to class 1.

For this worked example choose

$$t_c = 1.$$

Binary cross-entropy loss

$$E_c = -[t_c \ln \hat{y}_c + (1 - t_c) \ln(1 - \hat{y}_c)].$$

Since $t_c = 1$,

$$E_c = -\ln(0.7514) = \boxed{0.2859}.$$

Why the output delta becomes very simple

For sigmoid + BCE,

$$\delta_o^{(c)} = \hat{y}_c - t_c.$$

Therefore

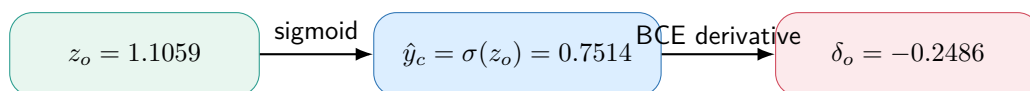
$$\delta_o^{(c)} = 0.7514 - 1 = \boxed{-0.2486}.$$

Interpretation of the sign. The target is 1, but the predicted probability is only 0.7514. The negative delta means gradient descent should increase the relevant output parameters so the next probability moves upward toward 1.

Output-layer gradients

$$\frac{\partial E_c}{\partial w_5} = \delta_o^{(c)} h_1 = (-0.2486)(0.5933) = \boxed{-0.1475},$$

$$\frac{\partial E_c}{\partial w_6} = (-0.2486)(0.5969) = \boxed{-0.1484}, \quad \frac{\partial E_c}{\partial b_3} = \boxed{-0.2486}.$$



Binary Classification - Backpropagate to the Hidden Layer

1. Backpropagate to hidden activations

$$\frac{\partial E_c}{\partial h_1} = \delta_o^{(c)} w_5 = (-0.2486)(0.40) = \boxed{-0.0995},$$

$$\frac{\partial E_c}{\partial h_2} = (-0.2486)(0.45) = \boxed{-0.1119}.$$

2. Hidden deltas

$$\delta_{h_1}^{(c)} = (-0.0995)(0.2413) = \boxed{-0.0240},$$

$$\delta_{h_2}^{(c)} = (-0.1119)(0.2406) = \boxed{-0.0269}.$$

3. Input-to-hidden gradients

Neuron h_1

$$\frac{\partial E_c}{\partial w_1} = -0.0240(0.05) = \boxed{-0.0012}$$

$$\frac{\partial E_c}{\partial w_2} = -0.0240(0.10) = \boxed{-0.0024}$$

$$\frac{\partial E_c}{\partial b_1} = \boxed{-0.0240}$$

Neuron h_2

$$\frac{\partial E_c}{\partial w_3} = -0.0269(0.05) = \boxed{-0.0013}$$

$$\frac{\partial E_c}{\partial w_4} = -0.0269(0.10) = \boxed{-0.0027}$$

$$\frac{\partial E_c}{\partial b_2} = \boxed{-0.0269}$$

The hidden-layer rule did not change. Once the output delta is known, the rest of backpropagation follows the same chain-rule structure:

$$\delta_{hj} = (\delta_o w_{j \rightarrow o}) h_j(1 - h_j).$$

Only the output delta formula changed because the output activation/loss pair changed.

Binary Classification - One Gradient-Descent Update

Use the same learning rate

$$\eta = 0.10.$$

Parameter	Old value	Gradient	New value
w_1	0.1500	-0.0012	0.1501
w_2	0.2000	-0.0024	0.2002
b_1	0.3500	-0.0240	0.3524
w_3	0.2500	-0.0013	0.2501
w_4	0.3000	-0.0027	0.3003
b_2	0.3500	-0.0269	0.3527
w_5	0.4000	-0.1475	0.4148
w_6	0.4500	-0.1484	0.4648
b_3	0.6000	-0.2486	0.6249

After the update, recompute the forward pass:

$$\hat{y}'_c = \boxed{0.7593}.$$

The BCE loss becomes

$$E'_c = -\ln(0.7593) = \boxed{0.2753}.$$

Learning direction

$$\hat{y}_c : 0.7514 \rightarrow 0.7593$$

$$E_c : 0.2859 \rightarrow 0.2753$$

The target is class 1, so the probability rises and the loss falls.

Why regression and classification move parameters in opposite directions here

Regression

$$\hat{y}_r = 1.1059 > t_r = 0.50$$

so

$$\delta_o^{(r)} > 0$$

and gradient descent decreases the main output parameters.

Classification

$$\hat{y}_c = 0.7514 < t_c = 1$$

so

$$\delta_o^{(c)} < 0$$

and gradient descent increases the main output parameters.

Regression vs. Classification - What Actually Changes?

	Regression	Binary classification
Shared hidden network	z_{h1}, z_{h2}, h_1, h_2 identical	z_{h1}, z_{h2}, h_1, h_2 identical
Output pre-activation	$z_o = 1.1059$	$z_o = 1.1059$
Output activation	Identity: $\hat{y}_r = z_o$	Sigmoid: $\hat{y}_c = \sigma(z_o)$
Target in this example	$t_r = 0.50$	$t_c = 1$
Loss	$\frac{1}{2}(\hat{y}_r - t_r)^2$	Binary cross-entropy
Output delta	$\delta_o^{(r)} = \hat{y}_r - t_r$	$\delta_o^{(c)} = \hat{y}_c - t_c$
Hidden backprop	Same chain-rule pattern	Same chain-rule pattern
Gradient formula	$\partial E / \partial w = \delta \times \text{local input}$	Same
Update rule	$\theta' = \theta - \eta \partial E / \partial \theta$	Same

The central rule to remember

Backprop does not need a new algorithm for every task.

The output activation/loss pair determines the first delta. After that, the same chain rule carries the sensitivity backward.

Backpropagation as a five-step recipe

1. **FORWARD** Compute every z and activation from left to right.
2. **LOSS** Compare the prediction with the target.
3. **OUTPUT DELTA** Differentiate the output activation/loss pair.
4. **BACKWARD** Send sensitivity through weights and activation derivatives.
5. **UPDATE** Apply $\theta' = \theta - \eta \nabla_{\theta} E$.

Common mistakes

- Using the wrong output delta for the chosen activation/loss pair.
- Forgetting the sigmoid derivative $h(1 - h)$ at hidden units.
- Multiplying a gradient by the wrong local input.
- Rounding too early and turning small real gradients into zero.
- Updating parameters before every gradient for the current example has been computed.

One-Page Reference Sheet

Shared forward equations

$$z_{h1} = w_1x_1 + w_2x_2 + b_1$$

$$h_1 = \sigma(z_{h1})$$

$$z_{h2} = w_3x_1 + w_4x_2 + b_2$$

$$h_2 = \sigma(z_{h2})$$

$$z_o = w_5h_1 + w_6h_2 + b_3$$

Classification head

$$\hat{y}_c = \sigma(z_o)$$

$$E_c = -[t \ln \hat{y} + (1 - t) \ln(1 - \hat{y})]$$

$$\delta_o^{(c)} = \hat{y}_c - t_c$$

Regression head

$$\hat{y}_r = z_o$$

$$E_r = \frac{1}{2}(\hat{y}_r - t_r)^2$$

$$\delta_o^{(r)} = \hat{y}_r - t_r$$

Shared backward equations

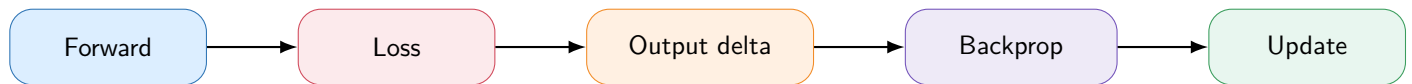
$$\frac{\partial E}{\partial w_5} = \delta_o h_1, \quad \frac{\partial E}{\partial w_6} = \delta_o h_2$$

$$\frac{\partial E}{\partial h_1} = \delta_o w_5, \quad \frac{\partial E}{\partial h_2} = \delta_o w_6$$

$$\delta_{h1} = \frac{\partial E}{\partial h_1} h_1(1 - h_1)$$

$$\delta_{h2} = \frac{\partial E}{\partial h_2} h_2(1 - h_2)$$

$$\frac{\partial E}{\partial w} = \delta \times \text{local input}$$



The network learns because the gradient tells every parameter how a small change would move the loss.